

AGENTIC AI -PROJECT

Project Summary / Abstract

Building an Agentic AI-powered autonomous campaign system, the system automatically discovers relevant targeted people like educational institutions, gathers information from public sources and APIs, generates personalized communication, manages email campaigns, tracks responses, performs intelligent follow-ups, and maintains campaign analytics.

Instead of conducting personalized outreach are mostly manual, time-consuming, and difficult to scale.

End Users/Target Audience

Engineering & degree colleges, autonomous institutions, universities, ECE,AI/ML,ECE aliened branches, training & placement cells, innovation incubation cells, official faculty coordinators that who are interested to learn ai course.

Project Features

Targeting Audience Info

Agents

Required Agents

- 1.Supervisor
- 2.planning
- 3.routing
- 4.memory
- 5.policy
- 6.monitoring
- 7.college discovery
- 8.website scrolling
- 9.department detection
- 10.opportunity
- 11.contact-discovering
- 12.lead qualification (trainer profile)

- 13.content generator
- 14.personalization
- 15.campaign manager
- 16.CRM Management
- 17.follow up
- 18.analytics agents.

End – End Workflow

Select states

Find colleges

Analyse official Website

Identify – AI/ML/ECE/CS Dept.

Prepare personalized proposal (give through the LLM with prompting)

Review/Approval campaign (by Human in the Loop)

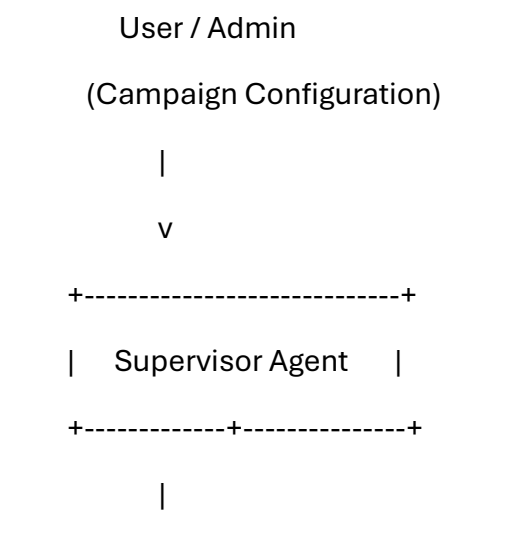
Send outreach through official contact channel

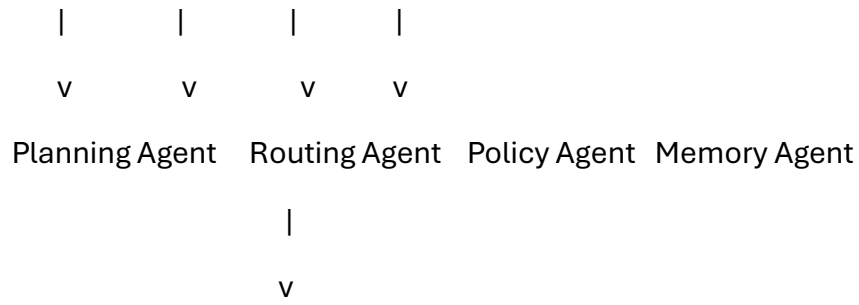
Track Responses

Schedule meeting

Analytics Dashboard

High-Level Architecture



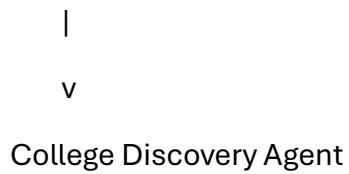


=====

COLLEGE DISCOVERY & LEAD GENERATION PIPELINE

=====

1. Select Target States



Find Engineering Colleges / Universities / Autonomous Institutions



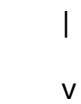
Website Scrolling Agent



Extract Official Website Information



Department Detection Agent



Identify

- AI / ML
- CSE
- ECE
- IT
- Data Science
- Allied Branches

|

v

Opportunity Detection Agent

|

v

Identify Training & Placement Cell

Innovation & Incubation Cell

Faculty Coordinators

|

v

Contact Discovery Agent

|

v

Collect Official Emails & Contacts

|

v

Lead Qualification Agent

|

v

Qualified Institutional Leads

=====

AI CONTENT GENERATION & CAMPAIGN EXECUTION

=====

|

v

Content Generator Agent

|

v

LLM + Prompt Engineering

|

v

Generate Personalized AI Course Proposal

|

v

Personalization Agent

|

v

Customize Proposal for Institution

|

v

=====

Human Review / Approval (HITL)

=====

|

v

Campaign Manager Agent

|

v

Official Outreach

(Email / Official Contact Channel)

=====

RESPONSE MANAGEMENT & CRM PIPELINE

=====

|

v

Track Responses

|

v

CRM Agent

|

+-----+

|

|

v

v

Follow-up Agent Meeting Scheduler

|

|

+-----+-----+

|

v

Analytics Agent

|

v

Campaign Dashboard & Reports

=====

SYSTEM OPERATIONS & GOVERNANCE

=====

Monitoring Agent

|

v

Workflow Monitoring

Error Detection

Agent Health

Performance Monitoring

Memory Agent

Stores campaign history, conversations,
institution profiles, and previous interactions.

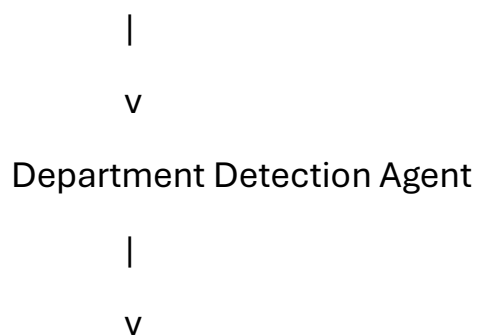
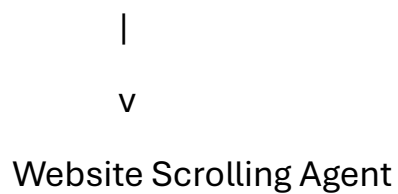
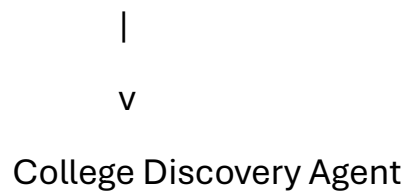
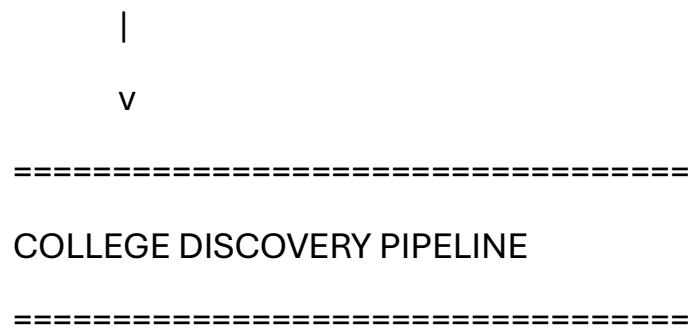
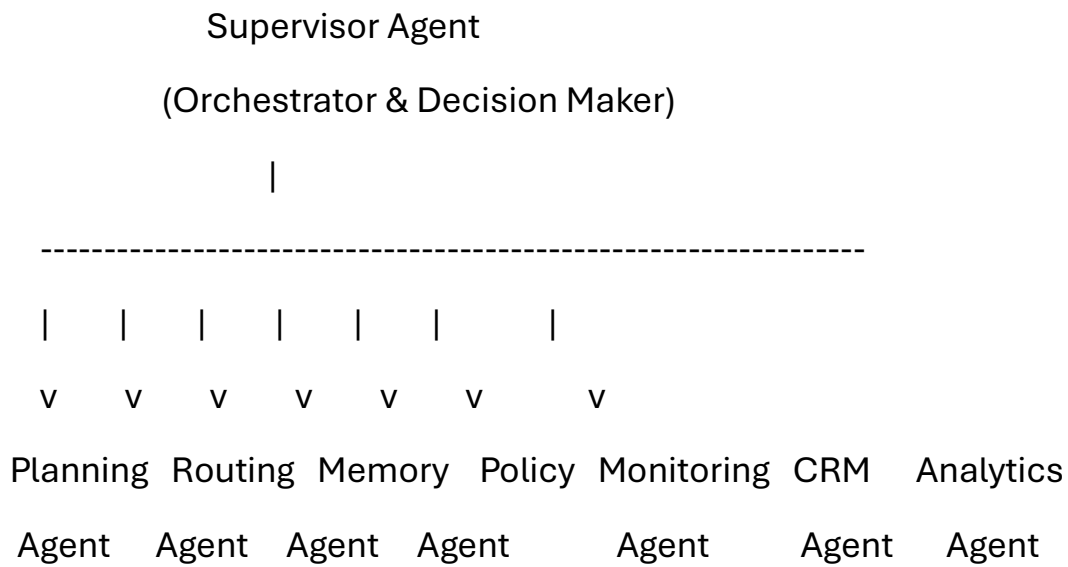
Policy Agent

Ensures campaign compliance,
email rules, approval policies,
and organizational guidelines.

Supervisor Agent

Coordinates all 18 agents,
handles task delegation,
and manages the complete workflow.

Agent Architecture



Opportunity Detection Agent

|

v

Contact Discovery Agent

|

v

Lead Qualification Agent

|

v

Content Generator Agent

|

v

Personalization Agent

|

v

Campaign Manager Agent

|

v

Email / Outreach Channel

|

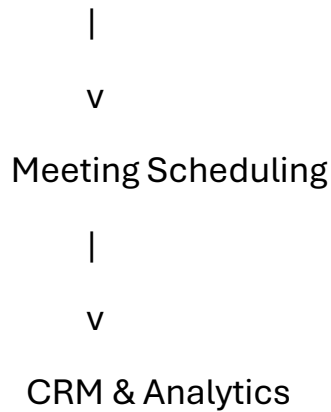
v

Recipient Responses

|

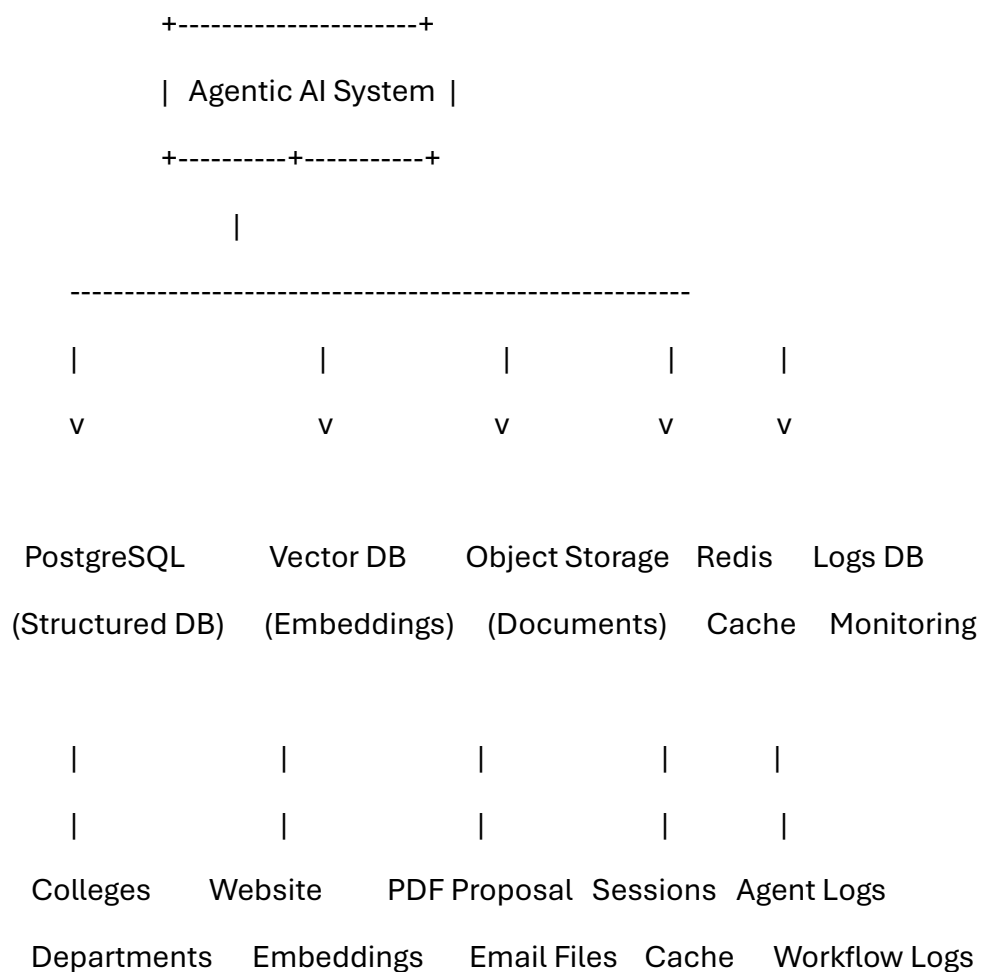
v

Follow-up Agent



Database Architecture

The system uses a **hybrid database architecture** because different types of data require different storage mechanisms. Structured data (colleges, contacts, campaigns) is stored in a relational database, while AI memory, documents, and semantic search are handled using vector storage and document storage.



Contacts Faculty Profiles Attachments State Errors

Campaigns Conversation

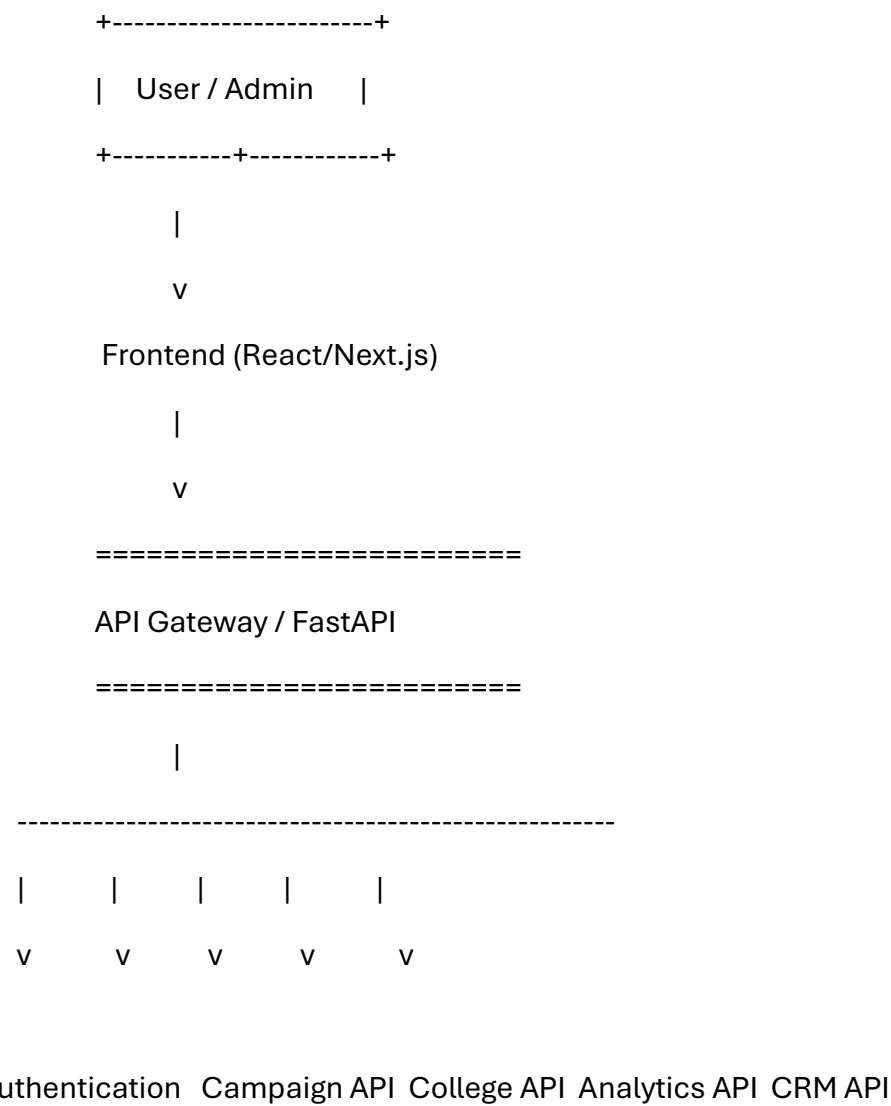
Responses Memory

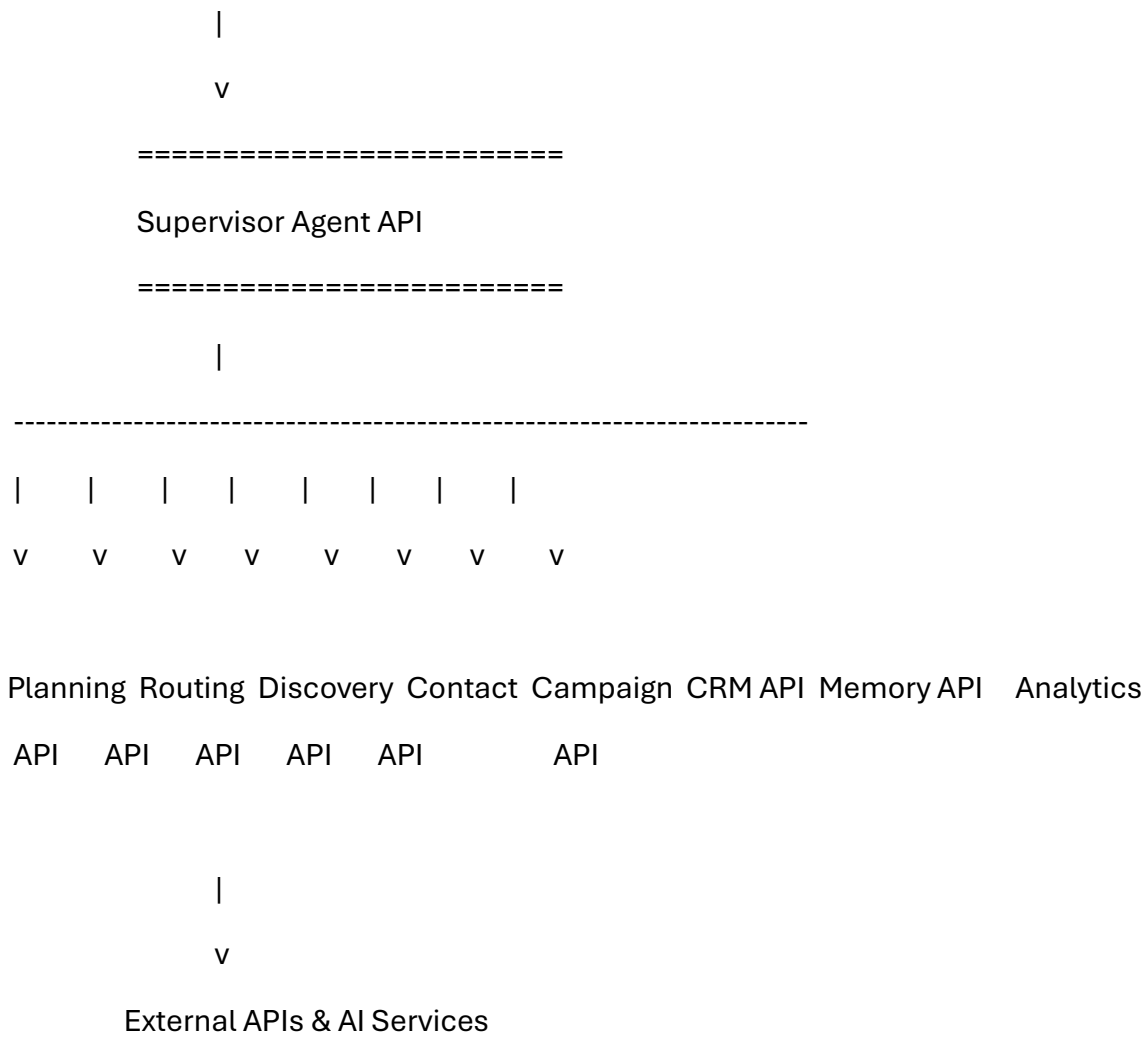
CRM

Analytics

API Architecture

The system follows an **API-first architecture**, where every agent communicates through well-defined APIs. External APIs provide institutional data, while internal APIs enable communication between agents, databases, LLMs, CRM, and campaign services.





Project Folder Structure

agentic-ai-outreach-system/

```
|
├── README.md
├── requirements.txt
├── .env
├── .gitignore
├── docker-compose.yml
└── Dockerfile
```

```
└─ pyproject.toml
|
└─ app/
| |
| | └─ main.py          # FastAPI Entry Point
| | └─ config.py
| | └─ settings.py
| | └─ dependencies.py
| |
| | └─ api/
| | | └─ routes/
| | | | └─ auth.py
| | | | └─ campaign.py
| | | | └─ colleges.py
| | | | └─ contacts.py
| | | | └─ crm.py
| | | | └─ analytics.py
| | | | └─ meetings.py
| | | | └─ health.py
| | |
| | | └─ middleware/
| | | └─ schemas/
| | | └─ dependencies.py
| |
| | └─ agents/
| | |
| | | └─ supervisor/
```

```
| | | └─ supervisor.py
| | | └─ workflow.py
| | |
| | └─ planning/
| | | └─ planning_agent.py
| | |
| | └─ routing/
| | | └─ routing_agent.py
| | |
| | └─ memory/
| | | └─ memory_agent.py
| | |
| | └─ policy/
| | | └─ policy_agent.py
| | |
| | └─ monitoring/
| | | └─ monitoring_agent.py
| | |
| | └─ college_discovery/
| | | └─ discovery_agent.py
| | |
| | └─ website_scrolling/
| | | └─ website_agent.py
| | |
| | └─ department_detection/
| | | └─ department_agent.py
| | |
```

```
| | ├── opportunity_detection/
| | |   └── opportunity_agent.py
| | |
| | ├── contact_discovery/
| | |   └── contact_agent.py
| | |
| | ├── lead_qualification/
| | |   └── qualification_agent.py
| | |
| | ├── content_generator/
| | |   └── content_agent.py
| | |
| | ├── personalization/
| | |   └── personalization_agent.py
| | |
| | ├── campaign_manager/
| | |   └── campaign_agent.py
| | |
| | ├── crm/
| | |   └── crm_agent.py
| | |
| | ├── follow_up/
| | |   └── followup_agent.py
| | |
| | └── analytics/
| |     └── analytics_agent.py
| |
```

```
|  ├── llm/
|  |   ├── prompts/
|  |   |   ├── proposal_prompt.py
|  |   |   ├── personalization_prompt.py
|  |   |   ├── qualification_prompt.py
|  |   |   └── followup_prompt.py
|  |   |
|  |   ├── models.py
|  |   ├── embeddings.py
|  |   └── rag.py
|  |
|  ├── services/
|  |   ├── email_service.py
|  |   ├── calendar_service.py
|  |   ├── crawler_service.py
|  |   ├── search_service.py
|  |   ├── storage_service.py
|  |   ├── notification_service.py
|  |   └── analytics_service.py
|  |
|  ├── database/
|  |   ├── connection.py
|  |   ├── models/
|  |   ├── repositories/
|  |   ├── migrations/
|  |   └── seed/
|  |
```

- | |— vectorstore/
- | | |— qdrant.py
- | | |— embeddings.py
- | | |— retrieval.py
- | |
- | |— workflows/
- | | |— campaign_workflow.py
- | | |— followup_workflow.py
- | | |— discovery_workflow.py
- | |
- | |— scheduler/
- | | |— email_scheduler.py
- | | |— followup_scheduler.py
- | | |— meeting_scheduler.py
- | |
- | |— integrations/
- | | |— google/
- | | |— gmail/
- | | |— sendgrid/
- | | |— firecrawl/
- | | |— tavily/
- | | |— serpapi/
- | | |— openai/
- | | |— groq/
- | | |— gemini/
- | |
- | |— utils/

```
| | ├── logger.py
| | ├── constants.py
| | ├── helpers.py
| | ├── validators.py
| | └── exceptions.py
| |
| └── dashboard/
|     ├── analytics_dashboard.py
|     ├── charts.py
|     └── reports.py
|
├── data/
|   ├── raw/
|   ├── processed/
|   ├── embeddings/
|   ├── proposals/
|   └── backups/
|
├── tests/
|   ├── test_agents.py
|   ├── test_api.py
|   ├── test_database.py
|   ├── test_campaign.py
|   └── test_llm.py
|
└── docs/
    ├── architecture.md
```

- | |— api_documentation.md
- | |— database_design.md
- | |— workflow.md
- | |— deployment.md
- |
- |— scripts/
- | |— init_db.py
- | |— seed_data.py
- | |— crawl_colleges.py
- | |— run_campaign.py
- |
- |— logs/
- | |— application.log
- | |— agents.log
- | |— campaign.log
- |
- |— deployment/
- | |— docker/
- | |— kubernetes/
- | |— nginx/
- | |— github_actions/
- | |— terraform/